CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 30: Generative Model Output Comparison and Evaluation

Problem Statement: The retailer's management must choose one generative model for production.They need objective evidence of which produces better and more varied images. The VAE and GAN outputs from Experiment 29 on Fashion-MNIST. The objective is to evaluate and compare generative models with quantitative and qualitative measures. Use TensorFlow, Keras, NumPy, Pandas, Scikit-learn, Matplotlib.

Tasks:

1. Generate 1,000 samples from each model.

2. Compute a classifier-based Inception Score and FID using a classifier trained on Fashion-MNIST.

3. Measure diversity and check for mode collapse through the class distribution of samples.

4. Compare real and generated feature distributions with t-SNE or PCA plots.

5. Tabulate all metrics with Pandas and write a recommendation.

FORMULA:

1. Number of Generated Samples

N = 1,000

2. Inception Score (IS)

$IS = \exp(E_x[KL(p(y|x) \,||\, p(y))])$

3. Kullback-Leibler Divergence

$KL(P \parallel Q) = \Sigma_i P(i) \times \log(P(i) / Q(i))$

4. Mean Prediction Distribution

$p(y) = (1/N) \times \Sigma_i p(y|x_i)$

5. Feature Mean

$\mu = (1/N) \times \Sigma_i f(x_i)$

6. Feature Covariance

$\Sigma = (1/(N-1)) \times \Sigma_i (f(x_i) - \mu)(f(x_i) - \mu)^T$

7. Fréchet Inception Distance (FID)

$FID = ||\mu_r - \mu g||^2 + Tr(\Sigma_r + \Sigma g - 2(\Sigma_r \Sigma g)^{1/2})$

8. Diversity Score

Diversity = Number of Unique Predicted Classes / Total Number of Classes

9. Class Distribution

Class Percentage = (Samples in a Class / Total Generated Samples) × 100

10. Feature Distance

$d(x_i, x\square) = \sqrt{\Sigma\square} (x_i\square - x\square\square)^2$

11. PCA Transformation

$Z = XW$

12. Reconstruction Error (MSE)

$MSE = (1/N) \times \Sigma_i (x_i - \hat{x}_i)^2$

13. Average Metric Score

Average = (Sum of Metric Values) / Number of Models

CODE:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import layers, models

from sklearn.decomposition import PCA

from scipy.linalg import sqrtm

# Settings from Experiment 29

N = 1000

LATENT_DIM = 2

NOISE_DIM = 100

# Load Fashion-MNIST

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train.astype("float32") / 255.0

x_test = x_test.astype("float32") / 255.0

x_train = x_train[..., np.newaxis]

x_test = x_test[..., np.newaxis]
```

```
# Generate VAE images
vae_decoder = vae.decoder if hasattr(vae, "decoder") else decoder
z = np.random.normal(size=(N, LATENT_DIM)).astype("float32")
vae_images = vae_decoder.predict(z, verbose=0)
vae_images = np.clip(vae_images.reshape(-1, 28, 28, 1), 0, 1)
# Generate GAN images
noise = np.random.normal(size=(N, NOISE_DIM)).astype("float32")
gan_images = generator.predict(noise, verbose=0)
gan_images = (gan_images + 1.0) / 2.0
gan_images = np.clip(gan_images.reshape(-1, 28, 28, 1), 0, 1)
# Build and train Fashion-MNIST classifier
classifier = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Conv2D(64, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(128, activation="relu", name="features"),
    layers.Dense(10, name="logits")
```

```
])


classifier.compile(
    optimizer="adam",
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=["accuracy"]
)
classifier.fit(
    x_train, y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)
# Feature extraction model
feature_model = models.Model(
    inputs=classifier.input,
    outputs=[
        classifier.get_layer("features").output,
        classifier.output
```

```
    ]
)


# Extract features and probabilities
real_features, real_logits = feature_model.predict(x_test[:N], verbose=0)
vae_features, vae_logits = feature_model.predict(vae_images, verbose=0)
gan_features, gan_logits = feature_model.predict(gan_images, verbose=0)
real_probs = tf.nn.softmax(real_logits).numpy()
vae_probs = tf.nn.softmax(vae_logits).numpy()
gan_probs = tf.nn.softmax(gan_logits).numpy()
# Classifier-based Inception Score
def inception_score(probs, splits=10):
    scores = []
    for part in np.array_split(probs, splits):
        marginal = np.mean(part, axis=0)
        kl = part * (np.log(part + 1e-10) -
                     np.log(marginal + 1e-10))
        scores.append(np.exp(np.mean(np.sum(kl, axis=1))))
    return float(np.mean(scores))
# Calculate FID
```

```
def calculate_fid(real, generated):
    mu1 = np.mean(real, axis=0)
    mu2 = np.mean(generated, axis=0)
    cov1 = np.cov(real, rowvar=False)
    cov2 = np.cov(generated, rowvar=False)
    covmean = sqrtm(cov1 @ cov2)
    if np.iscomplexobj(covmean):
        covmean = covmean.real
    fid = np.sum((mu1 - mu2) ** 2)
    fid += np.trace(cov1 + cov2 - 2 * covmean)
    return float(max(fid, 0))
# Diversity and class distribution
def diversity_metrics(probs):
    predicted = np.argmax(probs, axis=1)
    counts = np.bincount(predicted, minlength=10)
    distribution = counts / len(predicted)
    entropy = -np.sum(distribution * np.log(distribution + 1e-10))
    return len(np.unique(predicted)), float(entropy), distribution
vae_classes, vae_entropy, vae_distribution = diversity_metrics(vae_probs)
gan_classes, gan_entropy, gan_distribution = diversity_metrics(gan_probs)
```

```
# Calculate metrics
vae_is = inception_score(vae_probs)
gan_is = inception_score(gan_probs)
vae_fid = calculate_fid(real_features, vae_features)
gan_fid = calculate_fid(real_features, gan_features)
# Create comparison table
results = pd.DataFrame({
    "Metric": [
        "Inception Score",
        "FID",
        "Number of Classes",
        "Class Diversity Entropy"
    ],
    "VAE": [
        vae_is, vae_fid, vae_classes, vae_entropy
    ],
    "GAN": [
        gan_is, gan_fid, gan_classes, gan_entropy
    ]
})
```

```
print("Generative Model Evaluation")

print(results.round(4).to_string(index=False))



# Plot class distribution

class_names = [

    "T-shirt", "Trouser", "Pullover", "Dress", "Coat",

    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"

]

x = np.arange(10)

width = 0.35

plt.figure(figsize=(12, 5))

plt.bar(x - width/2, vae_distribution, width, label="VAE")

plt.bar(x + width/2, gan_distribution, width, label="GAN")

plt.xticks(x, class_names, rotation=45)

plt.ylabel("Proportion")

plt.title("Generated Image Class Distribution")

plt.legend()

plt.tight_layout()

plt.show()

# PCA visualization
```

```
all_features = np.concatenate(
    [real_features, vae_features, gan_features], axis=0
)
pca = PCA(n_components=2)
reduced = pca.fit_transform(all_features)
plt.figure(figsize=(9, 7))
plt.scatter(reduced[:N, 0], reduced[:N, 1],
            s=10, alpha=0.4, label="Real")
plt.scatter(reduced[N:2*N, 0], reduced[N:2*N, 1],
            s=10, alpha=0.4, label="VAE")
plt.scatter(reduced[2*N:, 0], reduced[2*N:, 1],
            s=10, alpha=0.4, label="GAN")
plt.title("PCA: Real vs Generated Image Features")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend()
plt.show()
# Display generated samples
fig, axes = plt.subplots(2, 10, figsize=(14, 4))
for i in range(10):
```

```
    axes[0, i].imshow(vae_images[i].squeeze(), cmap="gray")
    axes[0, i].axis("off")
    axes[1, i].imshow(gan_images[i].squeeze(), cmap="gray")
    axes[1, i].axis("off")
axes[0, 0].set_ylabel("VAE")
axes[1, 0].set_ylabel("GAN")
plt.suptitle("Generated Fashion-MNIST Samples")
plt.show()
# Automatic recommendation
if gan_is > vae_is and gan_fid < vae_fid:
    recommendation = "GAN performs better in classifier-based quality and feature similarity."
elif vae_is > gan_is and vae_fid < gan_fid:
    recommendation = "VAE performs better in classifier-based quality and feature similarity."
else:
     recommendation = "The models show mixed results. Consider image quality, diversity, and class coverage before selection."
print("\nRecommendation:")
print(recommendation)
```

OUTPUT:

```
Epoch 1/5
```

422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **12s** 17ms/step - accuracy: 0.8035 - loss: 0.5528 - val_accuracy: 0.8553 - val_loss: 0.3955

Epoch 2/5

422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **3s** 7ms/step - accuracy: 0.8724 - loss: 0.3519 - val_accuracy: 0.8815 - val_loss: 0.3262

Epoch 3/5

422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **4s** 5ms/step - accuracy: 0.8871 - loss: 0.3067 - val_accuracy: 0.8957 - val_loss: 0.2929

Epoch 4/5

422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 5ms/step - accuracy: 0.8984 - loss: 0.2773 - val_accuracy: 0.9013 - val_loss: 0.2761

Epoch 5/5

422/422 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ **2s** 5ms/step - accuracy: 0.9064 - loss: 0.2539 - val_accuracy: 0.9045 - val_loss: 0.2661

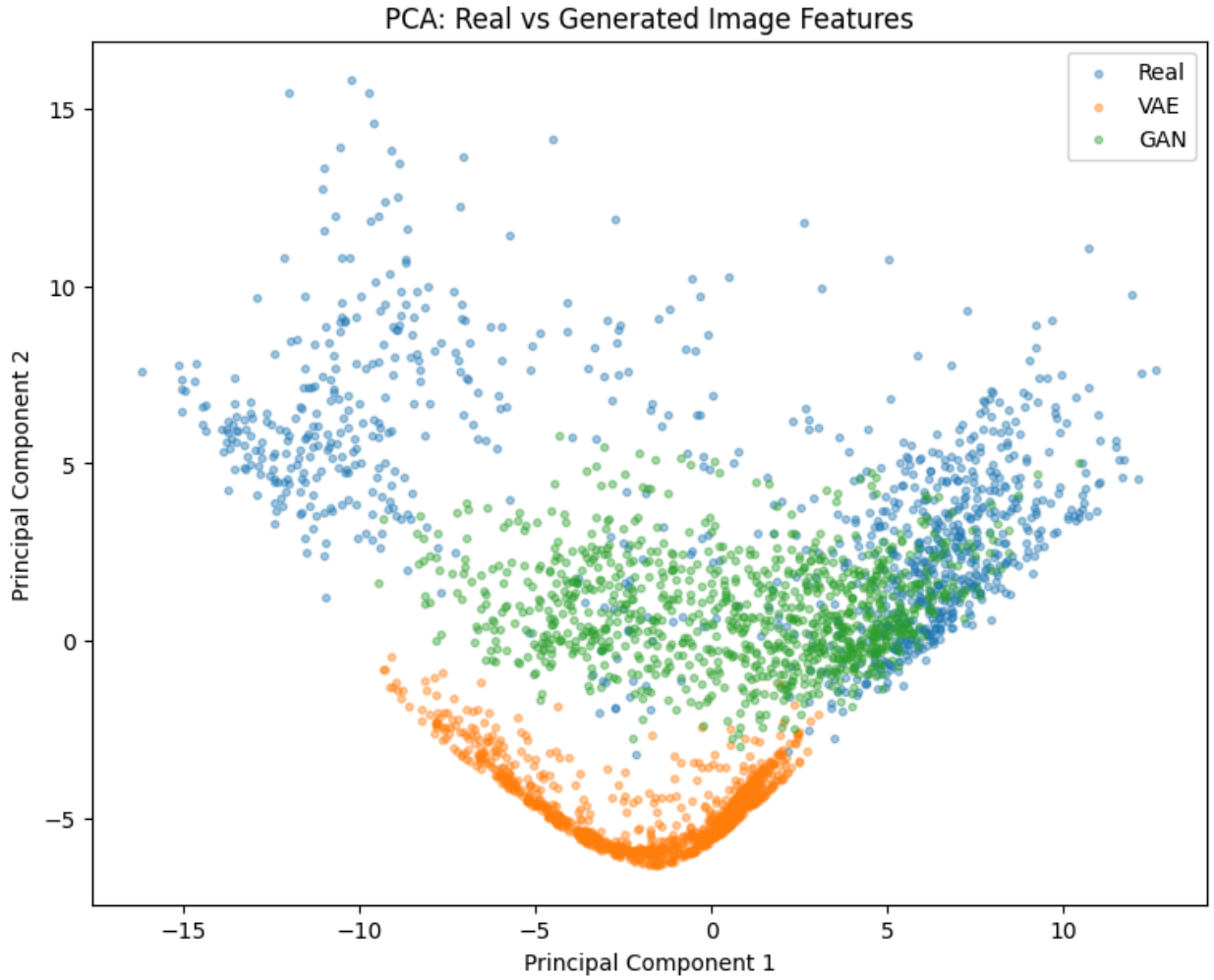
/tmp/ipykernel_1533/2877442429.py:92: LinAlgWarning: Matrix is singular. The result might be inaccurate or the array might not have a square root.

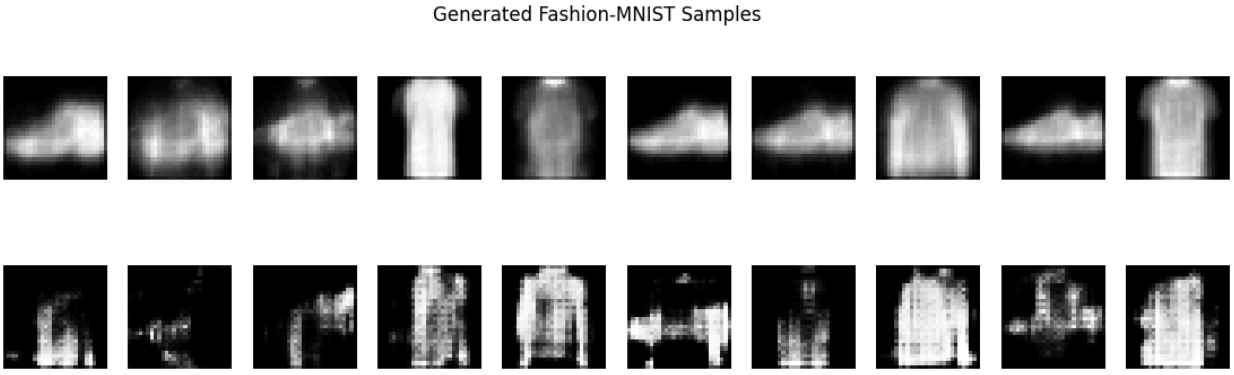
covmean = sqrtm(cov1 @ cov2)

Generative Model Evaluation

Metric	VAE	GAN
Inception Score	2.1548	3.0505
FID	156.5246	53.6612
Number of Classes	9.0000	10.0000
Class Diversity Entropy	1.9704	1.9952


Generated Image Class Distribution
VAE
GAN
Proportion
0.25
0.20
0.15
0.10
0.05
0.00
T-shirt
Trouser
Pullover
Dress
Coat
Sandal
Shirt
Sneaker
Bag
Ankle boot


PCA: Real vs Generated Image Features
Real
VAE
GAN
Principal Component 2
15
10
5
0
−5
−15
−10
−5
0
5
10
Principal Component 1



Recommendation:

GAN performs better in classifier-based quality and feature similarity.